Example Application
Security Assessment Report

Target application:	Example Application
Artifact path:	[FILL: path or repo reference]
Technology stack:	[FILL: e.g. Java/Spring MVC, MyBatis, Tomcat]
Review date:	2030-01-01
Review type:	Static / read-only white-box (source + byte-code)
Total findings:	[FILL: N (n Critical, n High, n Medium, n Low/Info)]
Classification:	CONFIDENTIAL

Confidentiality notice: [FILL: State plainly if this report contains real production secrets found verbatim in the artifact - if so, treat this document as strictly confidential and rotate every credential listed immediately. All Proof-of-Concept requests are illustrative only unless stated otherwise.]

[FILL: State the review constraints, e.g. "Static analysis only - no code modified, no network requests made, no application executed."]

Contents

I Security Assessment **2**

1 Executive Summary **3**
1.1 Severity Tally . . . 3
1.2 Top Risks — Fix First . . . 3
1.3 Report Structure . . . 4

2 Scope and Methodology **5**
2.1 Review Scope . . . 5
2.2 Methodology . . . 5
2.3 Confidence Levels . . . 6
2.4 Limitations . . . 6

3 Injection **7**
3.1 F-001 — `[FILL: Short, specific finding title]` . . . 7
3.2 F-002 — `[FILL: Second finding title in this category]` . . . 8

4 Broken Access Control **9**
4.1 F-010 — `[FILL: Short, specific finding title]` . . . 9
4.2 F-011 — `[FILL: Third finding title in this category]` . . . 9

5 Cross-Site Scripting **10**
5.1 F-020 — `[FILL: Short, specific finding title]` . . . 10

6 Vulnerable Dependencies **11**
6.1 F-030 — `[FILL: Short finding title, e.g. "End-of-Life Framework Version with Known RCE"]` . . . 11

II Remediation and Verification **12**

7 Prioritized Remediation Roadmap **13**
7.1 P0 — Fix Immediately (Unauthenticated, High Impact) . . . 13
7.2 P1 — Short-Term (Authenticated / Moderate Impact) . . . 13
7.3 P2 — Hardening . . . 13

A Complete Finding Matrix **14**

B Evidence File and Class Index **16**
B.1 Configuration Files 16
B.2 Configuration / Secret-Bearing Files 16

C Runtime Verification Checklist **17**
C.1 Ground Rules (Read First) 17
C.2 1 — Deployment and Configuration (Non-exploit, Server Read-only) . . 17
C.3 2 — Finding-by-Finding Confirmation 17

Part I

Security Assessment

Chapter 1

Executive Summary

[FILL: One or two sentences stating what this report does and does not cover - e.g. "this report identifies vulnerabilities only; no fixes were applied and no remediation changes were made to the codebase."]

[FILL: One paragraph summarising the overall posture: the categories of weakness found, and the worst realistic outcome an unauthenticated or low-privileg attacker could achieve.]

1.1 Severity Tally

The assessment produced [FILL: N] **findings** ([FILL: F-001 through F-0NN]) across [FILL: n] review passes.

Table 1.1: Finding count by severity

Severity	Count	Finding IDs
Critical	[FILL: n]	[FILL: F-001, F-002, ...]
High	[FILL: n]	[FILL: F-003, ...]
Medium	[FILL: n]	[FILL: F-004, ...]
Low / Info	[FILL: n]	[FILL: F-005, ...]
Total	[FILL: N]	

1.2 Top Risks — Fix First

[FILL: One sentence introducing the list below.]

1. [FILL: Headline risk cluster 1] (CRITICAL — §3, sec:F-001).
2. [FILL: Headline risk cluster 2] (HIGH — §4, sec:F-010).
3. [FILL: Headline risk cluster 3] (CRITICAL / HIGH — §6).

1.3 Report Structure

Part I (§§3–6)
: Finding-by-finding write-up with descriptions, Proofs of Concept (illustrative), impacts, and mitigations, organized by vulnerability category.

Part II (§7)
: Prioritized remediation roadmap (P0 immediate / P1 short-term / P2 hardening).

Appendix A
: Complete finding matrix with ID, severity, confidence, and category.

Appendix B
: Evidence file-and-class index mapping each category to its key source locations.

Appendix C
: Runtime verification checklist for authorized testers to confirm/deny each finding.

Chapter 2

Scope and Methodology

2.1 Review Scope

In scope [FILL: Enumerate what was reviewed - e.g. source tree, compiled bytecode, config files, dependency manifests - with rough size (file/class counts) if useful context.]

- [FILL: e.g. Views / templates (N files)]
- [FILL: e.g. Web/security XML or annotation-based configuration]
- [FILL: e.g. Properties / environment configuration files]
- [FILL: e.g. Compiled classes, analyzed via bytecode disassembly (source was absent)]

Out of scope [FILL: e.g. Dynamic testing; running the application; network calls; binary data files. No code was modified.]

Context path [FILL: https://example.com/app (used in illustrative PoCs only - not executed).]

2.2 Methodology

The assessment followed a static, read-only white-box approach:

1. **Configuration analysis** — [FILL: web/security config, properties files, ORM mapping config.]
2. **Secret scanning** — all configuration and selected constant pools scanned for credentials, keys, and tokens.
3. **Authorization model analysis** — [FILL: the framework's auth filter chain and any public-endpoint allowlist.]
4. **Input/output handling analysis** — [FILL: templates/views scanned for unescaped output, raw query construction.]
5. **Bytecode / binary analysis** — [FILL: if source was unavailable, note the disassembly tool used and what was searched for (injection sinks, crypto primitives, PRNG use).]
6. **Dependency audit** — library version fingerprinting against known CVE databases.

2.3 Confidence Levels

Each finding is assigned one of two confidence levels:

Confirmed Direct evidence quoted from source or bytecode — the vulnerability exists as described.

Suspected The mechanism is present but the full taint path or runtime-derived data could not be verified statically. These require dynamic confirmation (see Appendix C).

2.4 Limitations

- `[FILL: Note any part of the codebase that could not be verified statically (e.g. runtime DB-driven authorization tables, dynamic includes).]`
- `[FILL: Note any deployment/config-dependent finding whose real-world severity depends on something only knowable on the live server (see Appendix C).]`

Chapter 3

Injection

Systemic context: [FILL: If this category has a systemic root cause - e.g. a shared data-access helper that never parameterises input - describe it here once, so each finding below can just reference it instead of repeating the explanation.]

3.1 F-001 — [FILL: Short, specific finding title]

F-001 —

Severity: CRITICAL **Confidence:** Confirmed

Location: [FILL: package/Class.java]; reachable from [FILL: endpoint(s) or entry point(s)].

Description: [FILL: Explain, in plain language, what untrusted input reaches the vulnerable sink and why it is not safely handled.]

```
-- minimal illustrative SQL showing the pattern -- replace with
    your own, not real table/column names
```

Listing 3.1: [FILL: Illustrative injectable query - no real schema/data]

Proof of Concept (illustrative — not executed):

```
# illustrative request only -- replace with your own, no real
    hostnames/tokens/credentials
```

Listing 3.2: [FILL: PoC title]

Impact: [FILL: One or two sentences: what an attacker can concretely do as a result.]

Remediation: [FILL: The specific fix, e.g. "use parameterised queries / bind variables everywhere this pattern occurs."]

3.2 F-002 — [FILL: Second finding title in this category]

F-002 — [FILL: Second finding title]

Severity: MEDIUM **Confidence:** Suspected

Location: [FILL: package/Class.java]

Description: [FILL: Describe the weaker/partial version of the same issue class.]

Impact: [FILL: ...]

Remediation: [FILL: ...]

Chapter 4

Broken Access Control

4.1 F-010 — [FILL: Short, specific finding title]

F-010 — [FILL: Short, specific finding title]

Severity: HIGH Confidence: Confirmed

Location: [FILL: package/Class.java]; endpoint [FILL: METHOD /path].

Description: [FILL: Explain what authorization check is missing or bypassable, and how an object identifier is trusted from the request without an ownership check.]

Proof of Concept (illustrative — not executed):

```
# illustrative request only -- replace with your own
```

Listing 4.1: [FILL: PoC title]

Impact: [FILL: What data/records an attacker can read or modify that they should not be able to.]

Remediation: [FILL: e.g. add a per-object ownership check; enforce a default-deny authorization filter.]

4.2 F-011 — [FILL: Third finding title in this category]

F-011 — [FILL: Third finding title]

Severity: LOW Confidence: Confirmed

Description: [FILL: A minor hygiene gap in the same category.]
Remediation: [FILL: ...]

Chapter 5

Cross-Site Scripting

5.1 F-020 — [FILL: Short, specific finding title]

F-020 — [FILL: Short, specific finding title]

Severity: MEDIUM **Confidence:** Confirmed

Location: [FILL: template/view/path]; endpoint [FILL: METHOD /path].

Description: [FILL: Explain which output is rendered without encoding, and under what condition an attacker controls it (reflected/stored).]

```
<!-- minimal illustrative template snippet, replace with your
    own, not the real file contents -->
```

Listing 5.1: [FILL: Illustrative unescaped output]

Impact: [FILL: Session/token theft, UI redress, or other consequence in this application's context.]

Remediation: [FILL: e.g. output-encode by default in the templating engine; add a strict CSP without unsafe-inline.]

Chapter 6

Vulnerable Dependencies

[FILL: One paragraph on the overall dependency posture - e.g. how out of date the stack is and whether any end-of-life libraries carry known RCE CVEs.]

Table 6.1: End-of-life / vulnerable libraries identified

Library	Version found	Severity	Known issue
[FILL: library-name]	[FILL: x.y.z]	Critical	[FILL: CVE-YYYY-NNNNN - one-line description]
[FILL: library-name]	[FILL: x.y.z]	High	[FILL: CVE-YYYY-NNNNN - one-line description]

6.1 F-030 — [FILL: Short finding title, e.g. "End-of-Life Framework Version with Known RCE"]

F-030 —

Severity: CRITICAL Confidence: Confirmed

Location: [FILL: dependency manifest path / bundled jar]

Description: [FILL: Name the library, the version found, and the known CVE(s) affecting that version.]

Impact: [FILL: What the CVE allows (e.g. unauthenticated RCE).]

Remediation: [FILL: Upgrade to the fixed version; note any breaking changes to plan for.]

Part II

Remediation and Verification

Chapter 7

Prioritized Remediation Roadmap

This roadmap groups the findings into three priority tiers based on exploitability, attack complexity, authentication requirements, and business impact. The ordering within each tier follows the principle of *fix the key that opens all the other doors first*.

Before starting any remediation: `[FILL: If any finding exposed a live credential, say so here and cross-reference it - rotation should happen before or alongside every other fix, regardless of completion order.]`

7.1 P0 — Fix Immediately (Unauthenticated, High Impact)

These findings are exploitable today by an unauthenticated attacker and carry Critical severity.

P0-1. `[FILL: Action, e.g. "Rotate and externalize all secrets"]` (sec:F-001). `[FILL: One-line detail.]`

P0-2. `[FILL: Action, e.g. "Fix unauthenticated SQL injection in ..."]` (**sec:F-023**). `[FILL: One-line detail.]`

7.2 P1 — Short-Term (Authenticated / Moderate Impact)

P1-1. `[FILL: Action]` (sec:F-010). `[FILL: One-line detail.]`

P1-2. `[FILL: Action]` (sec:F-020). `[FILL: One-line detail.]`

7.3 P2 — Hardening

P2-1. `[FILL: Action, e.g. "Upgrade end-of-life dependencies"]` (sec:F-030).

P2-2. `[FILL: Action, e.g. "Add SAST/dependency scanning to CI"]`.

Appendix A

Complete Finding Matrix

All findings listed with ID, severity, confidence, category, and a brief title. Cross-references link to the detailed finding sections.

ID	Severity	Confidence	Category	Title
sec:F-001	Critical	Confirmed	Injection	[FILL: Title]
sec:F-002	Medium	Suspected	Injection	[FILL: Title]
sec:F-010	High	Confirmed	Broken Access Control	[FILL: Title]
sec:F-011	Low	Confirmed	Broken Access Control	[FILL: Title]
sec:F-020	Medium	Confirmed	Cross-Site Scripting	[FILL: Title]
sec:F-030	Critical	Confirmed	Vulnerable Dependencies	[FILL: Title]

Appendix B

Evidence File and Class Index

This appendix maps each vulnerability category to its primary source-evidence locations within the artifact. All paths are relative to the artifact root.

B.1 Configuration Files

File / Path	Role / Findings
[FILL: path/to/config]	[FILL: Role, plus finding cross-refs, e.g. sec:F-010]
[FILL: path/to/config]	[FILL: Role, plus finding cross-refs]

B.2 Configuration / Secret-Bearing Files

File	Secret / Finding
[FILL: path/to/properties]	[FILL: What kind of secret it holds - not the value - and the finding it maps to, e.g. sec:F-001]

Appendix C

Runtime Verification Checklist

This checklist was produced by static analysis only. None of these steps were executed. They are instructions for authorized testers. Run only with written authorization from the application/asset owner, against a **staging / non-production** instance that mirrors production configuration. See the ground rules below before proceeding.

C.1 Ground Rules (Read First)

- ☐ **Authorization:** Obtain written authorization before running any test.
- ☐ **Environment:** Use a staging / non-production instance. Do **not** run injection, deserialization, payment, or upload exploits against production.
- ☐ **Backups:** Snapshot the database and filesystem before destructive tests.
- ☐ **No live gadget payloads:** for any deserialization/RCE test, use a benign callback-only payload against an isolated collaborator host – never an RCE command payload on a shared or production system.
- ☐ **Rotate first:** any secret found in this report must be assumed compromised. Rotate it before these tests, then verify the application still functions.
- ☐ **Notify SOC/ops:** tests will generate alerts and log entries.
- ☐ **Placeholders:** `BASE=https://staging-host/app`; `COOKIE` = an authenticated session cookie where noted.

C.2 1 — Deployment and Configuration (Non-exploit, Server Read-only)

- ☐ `[FILL: e.g. Confirm whether upload directories are web-served (affects F-0NN severity).]`
- ☐ `[FILL: e.g. Confirm whether a static IV is used in every cryptographic call path.]`

C.3 2 — Finding-by-Finding Confirmation

- ☐ sec:F-001 — `[FILL: one-line confirmation step]`
- ☐ **sec:F-023** — `[FILL: one-line confirmation step]`